Predictive Equipment Failure

Industrial Pump Fleet — RUL Prediction System

Data Science Architect | CODVO.AI Take-Home Assignment

Candidate: Ani | Capgemini Technology Services

Presentation structure: 45 min architecture walkthrough + 15 min live inference

Agenda

Segment	Topic	Duration
1	Problem framing & architecture decision	5 min
2	Data understanding & EDA findings	8 min
3	Label engineering — censoring treatment	7 min
4	Feature engineering — physics-informed	8 min
5	Model architecture & why not deep learning	7 min
6	Validation strategy & results	5 min
7	Error analysis — where it fails	5 min
8	Live inference + Q&A	15 min

Problem Framing: The Decision and Rationale

What we are actually predicting

Given a pump's sensor history up to the present, estimate how many operating cycles remain before the next failure event — early enough and reliably enough to raise a work order before the breakdown occurs.

Four options were on the table

Option	Description	Decision
A — RUL Regression	Predict cycles to failure continuously	CHOSEN
B — Failure-window classification	Binary: fails within N cycles?	Output contract mismatch
C — Survival / hazard model	Model time-to-event with censoring	Insufficient power (72 failures)
D — Custom	Any justified alternative	No superior alternative found

Why piecewise-linear RUL?

- Raw RUL starts at (e.g.) 350 for a 350-cycle pump. No sensor signal separates a pump with 350 cycles left from one with 200 — they look identical in healthy state.
- Capping at RUL_MAX (empirically selected, typically 100-150 cycles) encodes the degradation onset: we only claim predictive value within the degradation window.
- This is standard practice in the PHM literature (NASA C-MAPSS benchmark) and maximises signal-to-noise in the training labels.

What we trade away: we never predict 'this pump has 400 cycles left.' We predict RUL_MAX for anything beyond the degradation window. This is honest — the data does not support finer discrimination.

Data Understanding & EDA Findings

Dataset at a glance

File	Content	Key Fact
train.csv	80 pumps, long format ~16,600 rows, 20 sensor columns, hourly sampling	
train_units.csv	One row per pump	72 failed (event=1), 8 censored (event=0)
sample_test_input.csv	Inference schema	Same columns, no event label, 2 pumps

EDA protocol — five phases

- Phase 1 — Data integrity: schema validation, sentinel value detection (–999), cycle continuity check, duplicate detection.
- Phase 2 — Signal quality classification: each channel assigned to dead / noise-dominant / operational-setpoint / degradation-signal tier using Spearman correlation with –RUL in last 30% of life.
- Phase 3 — Fleet degradation profiles: align all failed units to failure (x = cycles-to-failure), plot mean ± 1-std. Identifies degradation onset and informs RUL_MAX selection.
- Phase 4 — Censoring distributional test: Mann-Whitney U comparing censored vs failed units at matched cycle windows. Validates exclusion strategy.
- Phase 5 — Operating regime clustering: identifies multi-modal operating points requiring within-regime normalisation.

Key EDA finding: RUL_MAX is auto-selected by maximising mean |Spearman(sensor, RUL)| across all degradation-window observations. This is the empirical degradation onset point — where the fleet-average sensor reading begins to systematically deviate from baseline.

Label Engineering: Handling Censoring Correctly

The censoring problem — and why it matters

8 of 80 pumps are right-censored: monitoring stopped while the pump was still healthy. Their true failure time is unknown and lies AFTER the last recorded cycle.

WRONG (silently poisons labels): assigning RUL=0 at the last cycle of a censored pump tells the model that a healthy pump behaves identically to a failing one. This is a career-ending mistake on real reliability data.

Correct treatment

- Failed units (72): raw_rul = max_cycle – current_cycle, then capped at RUL_MAX. sample_weight = 1.0.
- Censored units (8): raw_rul = NaN. sample_weight = 0.0. Excluded from the supervised objective. Retained for feature normalisation (their early-life healthy readings are valid calibration data).
- Mann-Whitney U test: distributions of censored vs failed units are statistically indistinguishable at matched cycle windows ($p > 0.05$). Censoring is random (uninformative) — exclusion strategy is valid.

RUL_MAX selection logic

Grid search over candidate values 50-200 cycles. For each candidate, compute mean |Spearman(sensor, RUL)| across all degradation-window rows of failed units. The optimal RUL_MAX maximises this signal-to-noise ratio.

Label structure:

```
RUL label = min(max_cycle - current_cycle, RUL_MAX)
```

Feature Engineering: Physics-Informed, Not Kitchen-Sink

Design philosophy

Features encode what physically happens to a degrading centrifugal pump. We do not feed raw columns into the model and hope it discovers patterns. We build features that correspond to known degradation mechanisms.

Category	Features built	Physical mechanism
Rolling stats (4 windows × 6 stats)	480	Local level, volatility, noise increase
OLS slope + curvature (3 windows)	120	Drift rate and acceleration
EWMA level + deviation (3 alphas)	120	Noise-robust tracking, anomaly detection
Baseline z-score (own healthy baseline)	40	Unit-relative deviation, removes mfg. variation
Cross-sensor interactions	~10	Efficiency, pressure ratio, specific vibration
TOTAL candidates	~790	→ 150 selected via XGBoost gain + SHAP

Key feature: baseline z-score

For each sensor channel, we compute z = (current_value - unit_healthy_mean) / unit_healthy_std, where unit_healthy_mean is derived from the first 20 cycles of THAT unit's life. This removes unit-to-unit manufacturing variation and gives a pure condition signal: z=0 means 'behaving normally for this specific pump'; z=3 means '3 standard deviations above its own healthy baseline.'

Feature selection pipeline

- Step 1: Drop features with >30% NaN (insufficient history for short units).
- Step 2: Quick XGBoost fit — keep top 150 by 'gain' importance.
- Step 3: SHAP beeswarm validation — confirm retained features show physically sensible direction of effect.
- Step 4: Manual review — reject any feature that cannot be explained by a physical mechanism.

Model Architecture: Why Not Deep Learning

The architecture decision matrix

Consideration	Analysis	Verdict for this problem
Training set size	72 failed units × ~200 cycles = ~14k rows	Small for DL
Sequence length	Variable 20-400+ cycles per unit	Manageable but irregular
Interpretability	Client needs to defend maintenance decisions	DL black box is risky
Overfitting risk	72 sequences — LSTM memorises failure patterns	HIGH with DL
Inference speed	Must run in minutes on CPU	Both DL and GBDT acceptable

Chosen architecture: XGBoost + LightGBM ensemble

- XGBoost with PHM asymmetric loss: penalises late predictions more steeply. Directly optimises for business cost asymmetry.
- LightGBM with RMSE: complementary learner; fast SHAP computation; native support for sample weights.
- Ensemble: weighted average of both models. Weights (xgb_w, lgbm_w) optimised via Optuna across GroupKFold-5 CV.
- Unit-level RUL = median of last-5-cycle predictions. Robust to single-cycle sensor noise spikes at the final observation.

CNN/LSTM experiment: we did implement and test a 1D CNN for comparison. Its validation RMSE was higher than the ensemble for this fleet size, confirming that the bias-variance trade-off favours GBDT at n=72 training sequences.

PHM asymmetric loss function

```
L(d) = exp(-d/13) - 1 if d < 0 [early prediction — gentle slope]
L(d) = exp( d/10) - 1 if d > 0 [late prediction — steeper slope]
```

where d = predicted_RUL - true_RUL. A late prediction of +20 cycles accumulates e^2 - 1 = 6.4 penalty units. An early prediction of -20 cycles accumulates e^(20/13) - 1 = 3.7 penalty units. Late predictions are 73% more costly.

Validation Strategy: Leak-Proof, Unit-Level

The most common overfitting mistake in time-series ML

WRONG: Train on cycles 1-200 of pump X, test on cycles 201-300 of pump X. This is interpolation, not prediction. It looks great in validation and fails catastrophically in production.

Our approach: GroupKFold with k=5

- All cycles of a given pump appear in exactly ONE fold as test data.
- Feature engineering baselines are re-fitted on training units only (no baseline leakage from test unit's own healthy history).
- 80 pumps / 5 folds = ~16 test units per fold — sufficient for stable estimates.

Metric suite

Metric	Formula	Business meaning
RMSE	sqrt(mean(d^2))	Average error in cycles (~hours)
PHM Score	Sum of asymmetric losses	Primary: accounts for cost asymmetry
Within-30%	% units with |d| <= 30	Maintenance window achievable
Late prediction %	% units with d > 0	Risk exposure: missed failures
Precision@30	P(true RUL<30 | pred<30)	False alarm rate at alert threshold
Recall@30	P(pred<30 | true RUL<30)	Coverage: how many failures caught

Progressive accuracy analysis

We compute all metrics stratified by RUL bucket: [0-25], [25-50], [50-75], [75-100], [100-125]. Accuracy should monotonically improve as RUL approaches 0. This validates that features capture the degradation signal and gives the operations team a precise answer to: 'How far in advance can the system reliably alert us?'

Error Analysis: Where the Model Fails

Known failure modes (honest accounting)

Failure Mode	Root Cause	Mitigation in place
Short-lifetime units (<50 cycles)	Insufficient trend feature history	Minimum 10-cycle guard; flag in output
Late predictions on slowly-degrading units	Degradation onset later than RUL_MAX	Calibrated failure_prob_30 output
High prediction variance (pred_std)	Noisy sensor at final cycles	Median of last-5-cycle predictions
Sensor dropout units	Missing values corrupt features	Global baseline fallback; NaN fill
Infant mortality (<20 cycles to failure)	Different failure mode — no trend time	Flagged; requires separate model

What I would do with more time

- Survival model companion: Cox PH or Weibull AFT to provide credible intervals on remaining life — convert point estimates to planning ranges.
- Multi-task learning: jointly predict RUL and failure mode (bearing wear, impeller cavitation, seal degradation). Each has a distinct sensor signature that a shared feature representation can leverage.
- Conformal prediction: distribution-free, calibrated confidence intervals per unit. No parametric assumptions required.
- Physics-informed features: if H-Q pump curves are available, compute efficiency degradation from the pump's operating point relative to its design curve. This is the gold-standard feature for centrifugal pump prognostics.
- Anomaly detection pre-filter: Isolation Forest trained on healthy-only early-life data. Independent early-warning layer that does not depend on the RUL model.

Live Inference: Presentation Day Playbook

One-command execution

Run this command the moment the test file is received:

```
python predict.py --input .csv --output live_predictions.csv --shap
```

Output contract (what they will see)

```
unit_id, predicted_rul, failure_prob_30, health_index, prediction_std,
warning_level
```

The --shap flag generates a SHAP waterfall plot for the highest-risk unit, showing which features drove the low RUL prediction. This is the strongest possible demonstration of model interpretability.

What to narrate during inference

- Point out any RED-flagged units: 'This pump has predicted_rul=X cycles — failure_prob_30 is Y%, meaning there is a Y% chance it fails within 30 cycles.'
- Open the SHAP waterfall: 'The prediction is driven primarily by [top feature] — this is the baseline z-score for the vibration channel, meaning this pump is currently 2.8 standard deviations above its own healthy vibration baseline.'
- Acknowledge uncertainty: 'The prediction_std of Z cycles means our last-5 cycle predictions were consistent — this is a high-confidence alert.'

Pre-flight checklist (rehearse before the call)

- [ ] python train.py completes without errors
- [ ] python predict.py --input data/sample_test_input.csv exits cleanly in < 3 min
- [ ] submission/predictions/predictions.csv has correct columns
- [ ] SHAP plot renders (requires shap + matplotlib installed)
- [ ] All artefact files present in submission/model_artifacts/

Architecture Summary

Stage	Component	Key Decision
1. Ingest	data_loader.py	Auto-detect sensors; replace sentinel values; validate schema
2. Labels	label_engineer.py	Piecewise RUL; exclude censored (sample_weight=0)
3. Features	feature_engineer.py	790 candidates: rolling + trend + EWMA + baseline + cross-sensor
4. Selection	select_features_by_importance()	XGBoost gain → top 150; SHAP validation
5. Train	RULEnsemble.fit()	XGBoost (PHM loss) + LightGBM (RMSE); Optuna tuning
6. Validate	UnitLevelValidator	GroupKFold-5; no unit across train/test boundary
7. Calibrate	calibrate_failure_prob()	Isotonic regression for failure_prob_30
8. Infer	predict.py	Median of last-5 cycles; SHAP explanation; warning level

Total wall-clock time: train.py < 10 min on CPU (no GPU required). predict.py < 2 min on CPU for any fleet size comparable to the test set.

Reproducibility guarantee

- All random seeds pinned: numpy=42, python=42, xgboost=42, lightgbm=42.
- All dependencies version-pinned in requirements.txt.
- A clean clone + pip install -r requirements.txt + python train.py produces identical results.
- Artefact checksums can be verified against the submitted model_artifacts/ directory.

Thank you — questions welcome.